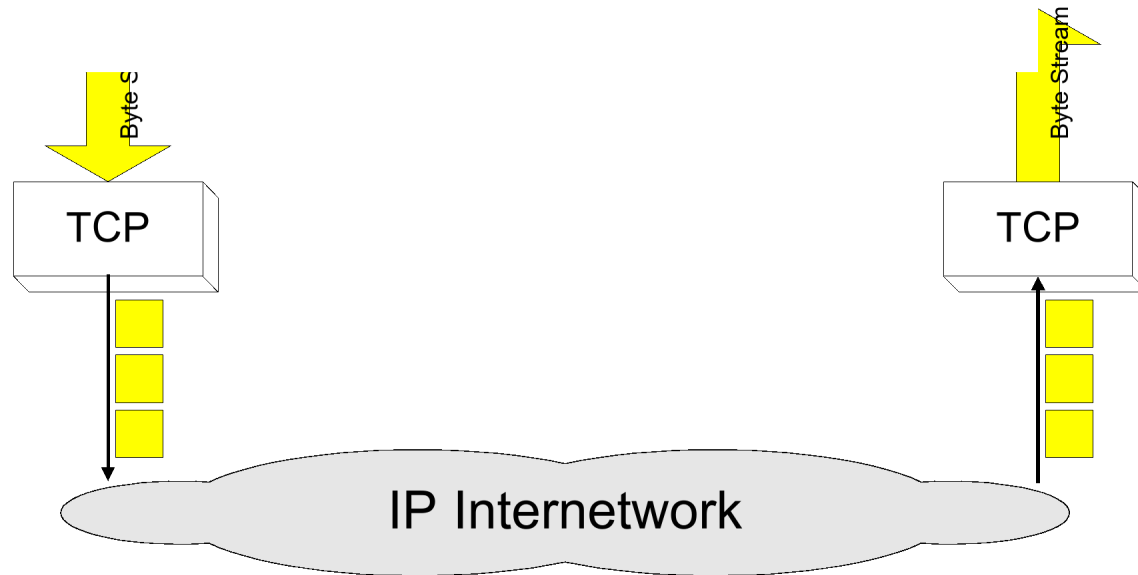

Relates to Lab 5. First module on TCP which covers packet format, data transfer, and connection management.

Overview

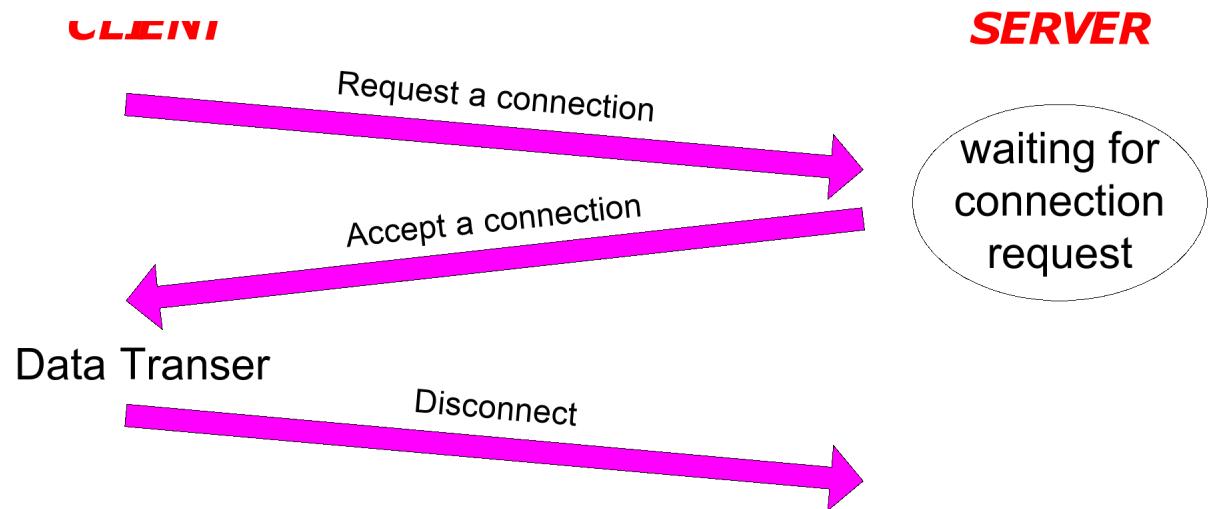
TCP = Transmission Control Protocol

- Connection-oriented protocol
- Provides a reliable unicast end-to-end byte stream over an



Connection-Oriented

- Before any data transfer, TCP establishes a **connection**:
 - One TCP entity is waiting for a connection (“**server**”)
 - The other TCP entity (“**client**”) contacts the server
-
- C
- Each con
full duplex



Reliable

- **Byte Stream Service:**

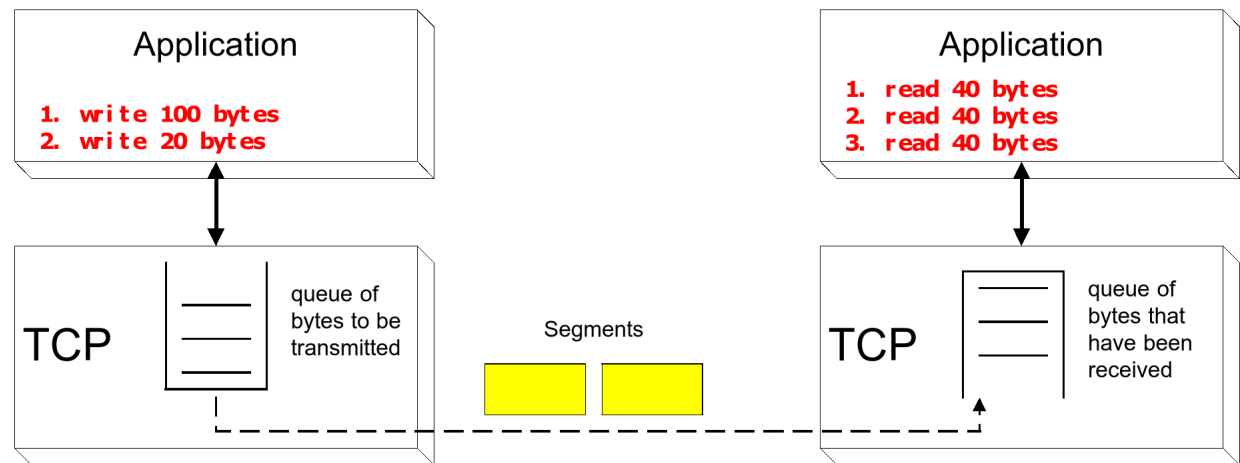
- At the sender, the byte stream from the application is broken up into chunks which are ca
- TCP sender maintains a timer. If an ACK is not received in time from the receiver, the segment is retransmitted

- **Detecting errors:**

- TCP uses a checksum to detect errors. Segments with invalid checksums are discarded
- Each byte that is transmitted has a sequence number

Byte Stream Service

- To the lower layers, TCP handles data in blocks - segments
- To the higher layers TCP handles data as a sequence of bytes and does not identify boundaries between bytes
-



TCP Packet Format

TCP segments have a 20-60 byte header with ≥ 0 bytes of application data

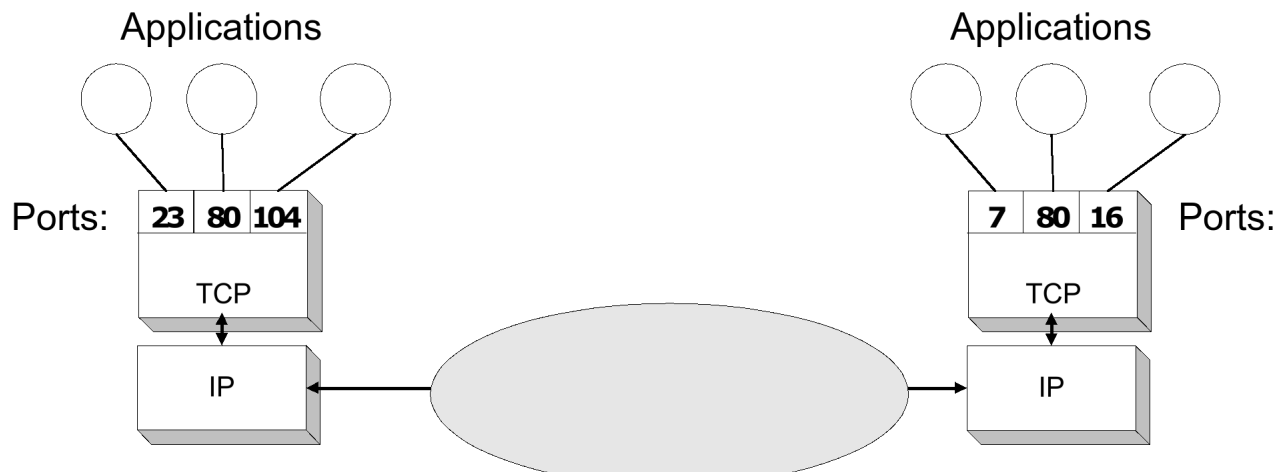
TCP Header																																	
Offsets	Octet	0								1								2					3										
Octet	Bit	0	1	2	3	4	5	6	7	8									19	20	21	22	23	24	25	26	27	28	29	30	31		
0																																	
4																																	
8	64																																
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S S Y	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if <i>data offset</i> > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

TCP header fields

- **Port Number:**

- A port number identifies the **process** associated with a connection.
- A pair **<IP address, port number>** identifies one

<server IP address, server port number> identify a **TCP connection**.



TCP header fields

- **Sequence Number (SeqNo):**

- Sequence number is 32 bits long.
- So the range of SeqNo is

stream

- An Initial Sequence Number (ISN) for a new connection is picked randomly at each end of the connection and is exchanged during connection establishment

TCP header fields

- **Acknowledgement Number (AckNo):**

- Acknowledgements can be piggybacked

- a segment from A -> B can contain an acknowledgement for data sent in the B -> A

ts

- if a h

K flag"

- The AckNo contains the **next** SeqNo that a receiving host is expecting to receive from the sender. Eg:

- The acknowledgement for a segment with sequence number "0" and data length of 1500 bytes is AckNo = 1500 (0-1499 = 1500 bytes, next byte is byte # 1500)
- The next segment sent from sender should have a SeqNo = 1500.

TCP header fields

- **Data Offset -> Header Length (4bits):**

- Length of header in units of 32-bit words (4bytes)

- **Note:** $2^4 = 16$ (0-15)

- max

- 40byte options field)

ue to the

- **Reserved (3bits):**

- Bits left open for future use. Set to ZERO

TCP header fields

- **Flag bits (9 x 1 bit control bits):**

- NS (1 bit) – ECN-nonce concealment protection (experimental: see RFC 3540). Proposal to prevent a receiver from hiding congested state from a sender.

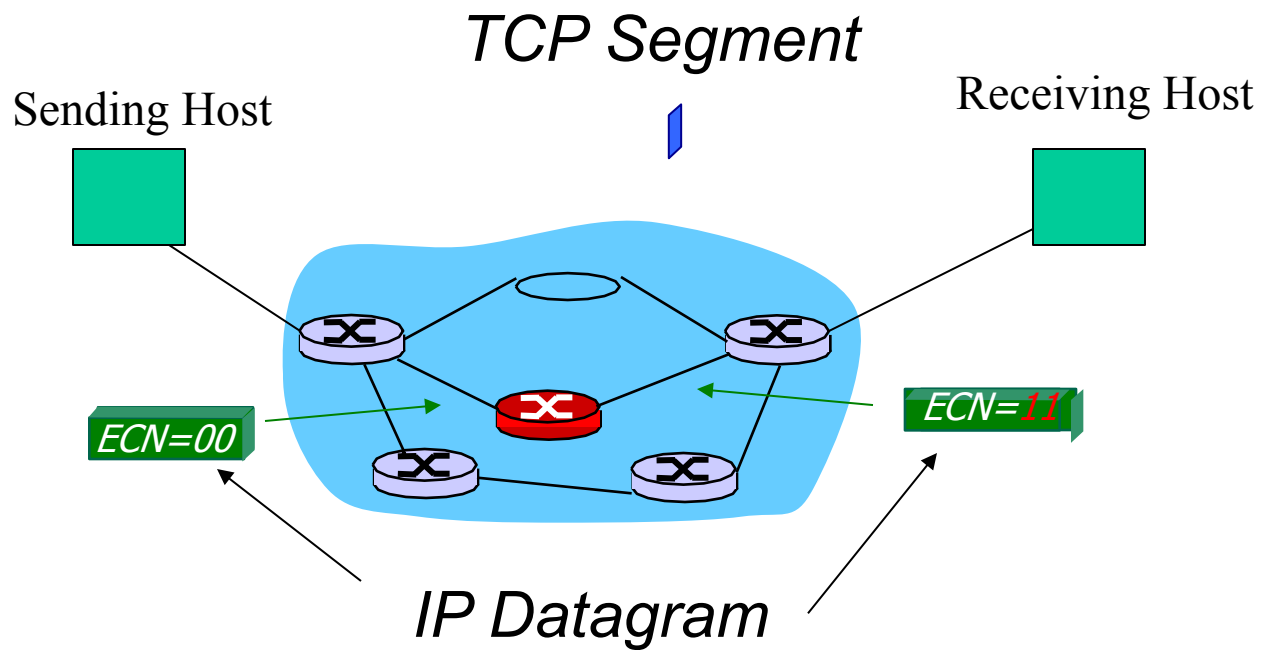
e
E

flag set

nism

- ECE (1 bit) – ECN-Echo has a dual role, depending on the value of the SYN flag. When ECE = 1, it indicates:
 - If the SYN flag is set (1) (i.e., a connection setup packet) -> the TCP peer is ECN capable. (Used during connection setup to indicate it will partake in congestion notification.)
 - If the SYN flag is clear (0) -> a packet with Congestion Experienced flag set (ECN=11) in IP header was received during normal transmission. This serves as an indication of network congestion (or impending congestion) to the TCP sender.

ECE and Congestion Notification



TCP header fields

- **Flag bits contd.:**

- URG (1 bit) – indicates that the Urgent pointer field is significant (has a value) and the following bytes contain an urgent message in the range:

SeqNo = 1st by

packets
flag set.

All
ould have this

- PSH (1 bit) – Push function. Sender asks receiver to push the buffered data to the receiving application. Normally set by sender when the sender's buffer is empty – has no more data to send for now.
- RST (1 bit) – Sender tells receiver to reset the connection. Receiver of a RST terminates the connection and indicates higher layer application about the reset.

TCP header fields

- **Flag bits contd:**

- SYN (1 bit) – Synchronize sequence numbers. Only the first packet sent from each end should have this flag set (during connectio

this set, and
others when it is clear.

- FIN (1 bit) – Last package from sender. Sender is done with transmitting
 - Used for closing a connection
 - Both sides of a connection must send a **FIN**
 - One side can be done before the other

TCP header fields

- **Window Size:**

- flow control from a receiver R limiting transmissions from a sender – “controlling flow by R of data being sent from S -> R”
- Each side of the connection maintains a receive window size in bytes

-

- Only valid if the urgent flag is set
- It indicates that the following bytes in the data field contain an urgent message:

SeqNo + urgent pointer = 1st byte of data field is start of urgent message

1st byte of “Normal” data starts at = SeqNo + urgent pointer

TCP header fields

- **TCP Checksum:**

- TCP checksum covers both **TCP Pseudo header** and TCP header and data. Pseudo header consists of:
 - IP Source address
 - IP Destination address

TCP lengt

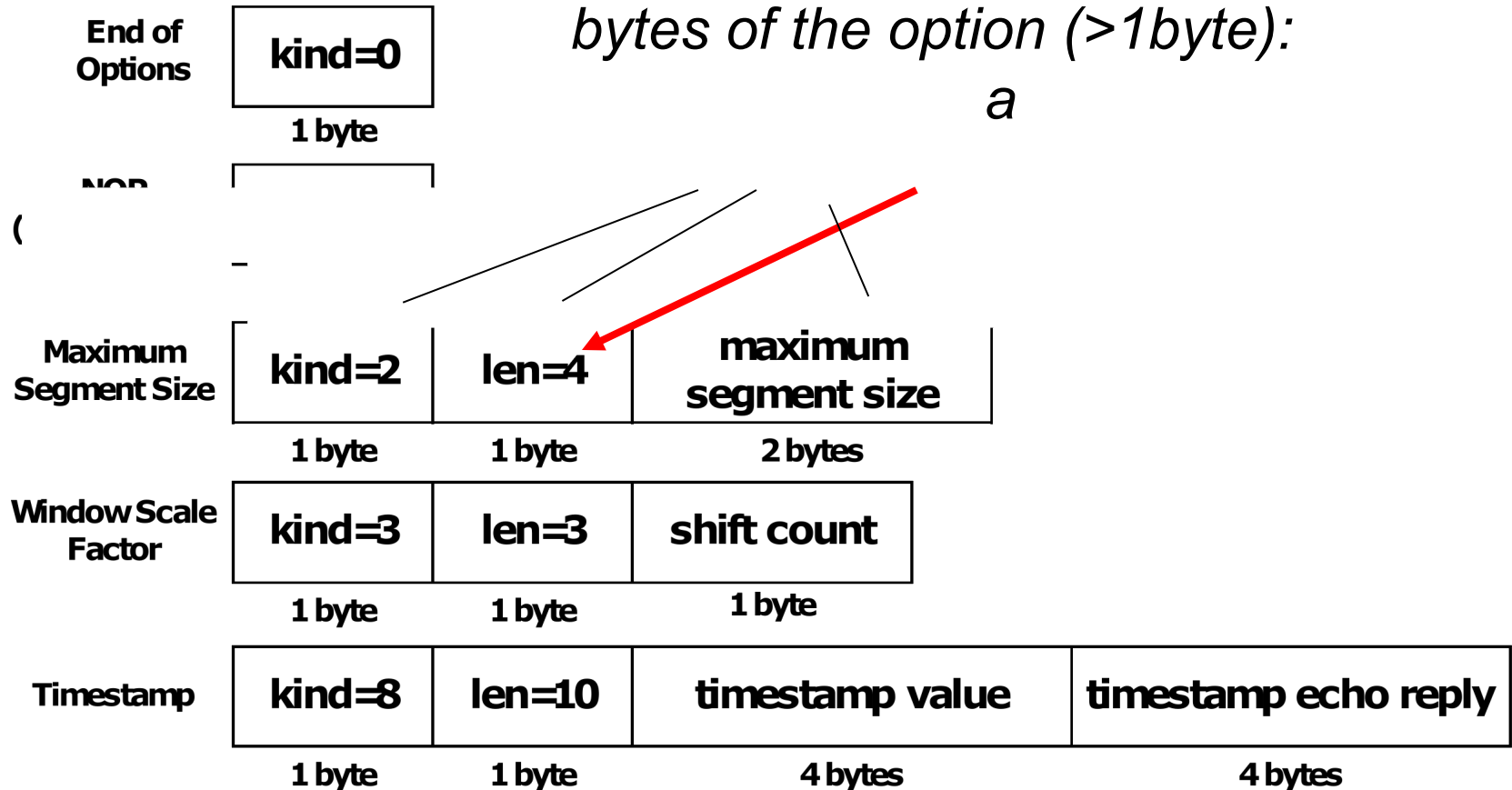
TCP pseudo-header for checksum computation (IPv4)				
Bit offset	0–3	4–7	8–15	16–31
0	Source address			
32	Destination address			
64	Zeros		Protocol	TCP length
96	Source port			Destination port
128	Sequence number			
160	Acknowledgement number			
192	Data offset	Reserved	Flags	Window
224	Checksum			Urgent pointer
256	Options (optional)			
256/288+	Data			

TCP header fields

- Some Options:

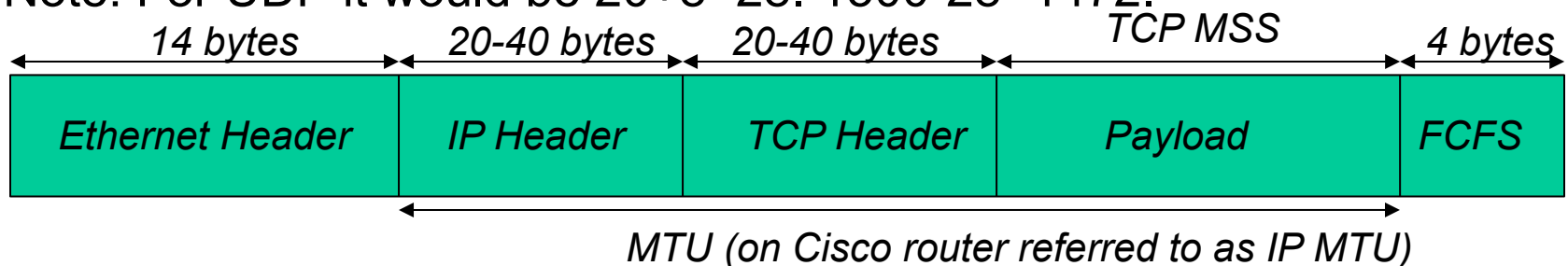
The “len” field indicates length in bytes of the option (>1byte):

a



MTU and MSS:

- The Maximum Transmission Unit (MTU) is the maximum length of data that can be transmitted by a protocol in one instance.
- Ethernet interface- the MTU size of an Ethernet interface is 1500 bytes by default
 - **excludes** the Ethernet trailer.
- F
 - 20 byte
 - $1500 - 40 = 1460$ byte
- 1460 is the maximum TCP payload that can be carried.
- This is what we refer to as TCP MSS. The diagram below visualizes this concept
- Note: For UDP it would be $20 + 8 = 28$. $1500 - 28 = 1472$.



Path Discovery Protocol

- Used for determining the maximum transmission unit (MTU) size on the network path between two Internet Protocol (IP) hosts, usually with the goal of avoiding **IP fragmentation**
- Path MTU Discovery *he Don't* *ts*
- *size of th* *an Internet*
Control Message Protocol (ICMP) *Fragmentation*
Needed (Type 3, Code 4)
- This will result in the source host reducing its MTU (via MSS) and trying again (trial and error).
- The process is repeated until the MTU is small enough to traverse each link on the entire path without fragmentation.

Connection Management in TCP

- **Opening a TCP Connection**
- **Closing a TCP Connection**
-
-

TCP Connection Establishment

- TCP uses a **three-way handshake** to open a connection:

(1) ACTIVE OPEN: Client sends a segment with

- SYN bit set
- port number of

(2) PASS

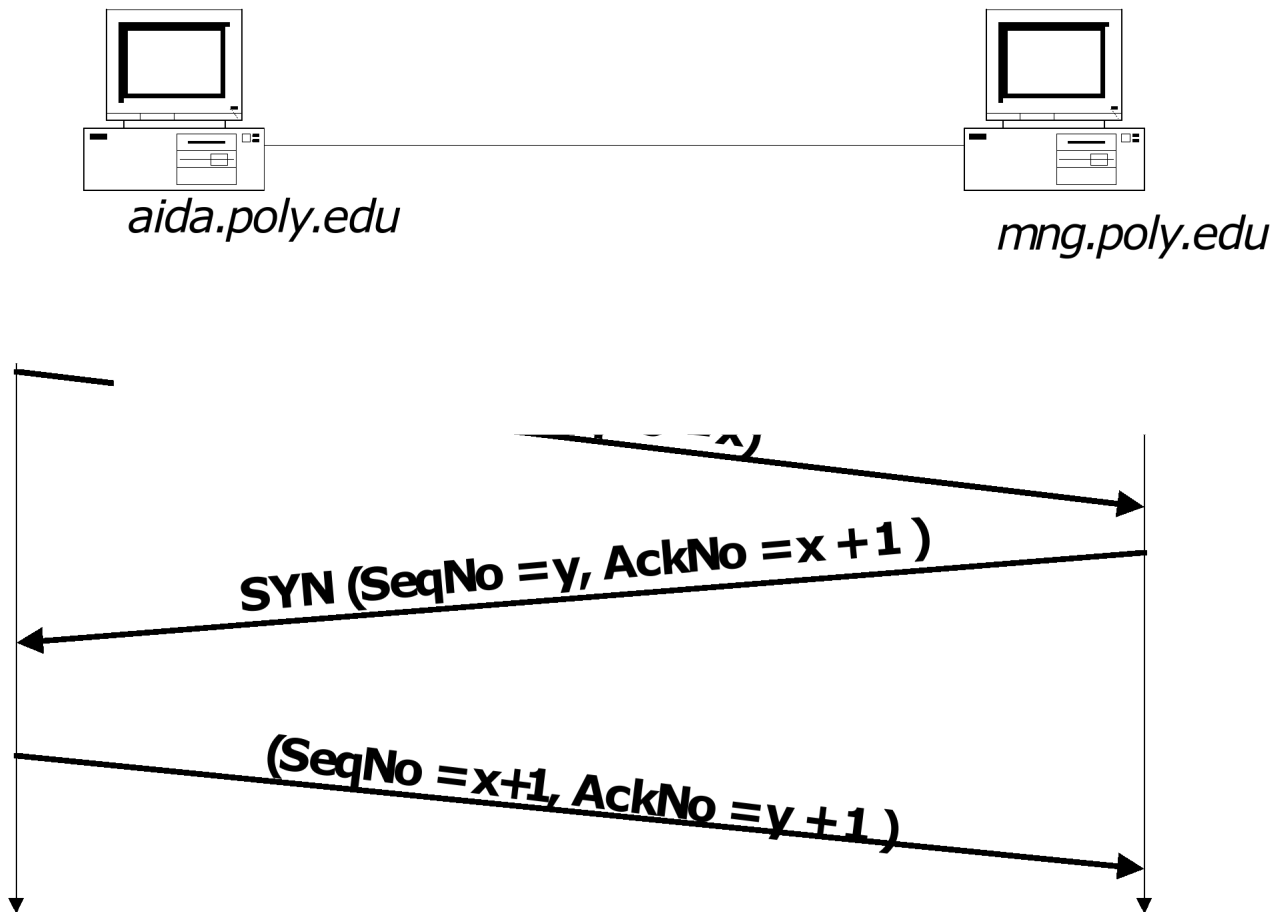
ent with

- SYN bit set
- initial sequence number of server, e.g. y
- ACK for ISN of client: $x+1$

(3) Client acknowledges by sending a segment with:

- SeqNo = $x+1$
- ACK ISN of server: $y+1$

Three-Way Handshake



A TCP SYN Packet

104	2703.569100	10.0.1.11	10.0.2.22	TCP	74	51336→7	[SYN] Seq=0 Win=29200 Len=0
105	2703.570099	10.0.2.22	10.0.1.11	TCP	74	7→51336	[SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
106	2703.570420	10.0.1.11	10.0.2.22	TCP	66	51336→7	[ACK] Seq=1 Ack=1 Win=29200 Len=0
107	2703.571851	10.0.1.11	10.0.2.22	ECHO	578		Request
108	2703.573041	10.0.2.22	10.0.1.11	TCP	66	7→51336	[ACK] Seq=1 Ack=513 Win=0 Len=0
109	2703.573120	10.0.2.22	10.0.1.11	ECHO	578		Response
110	2703.573416	10.0.1.11	10.0.2.22	TCP	66	51336→7	[ACK] Seq=513 Ack=513 Win=0 Len=0
111	2703.575219	10.0.1.11	10.0.2.22	TCP	66	51336→7	[FIN, ACK] Seq=513 Ack=513 Win=0 Len=0
112	2703.576755	10.0.2.22	10.0.1.11	TCP	66	7→51336	[FIN, ACK] Seq=513 Ack=513 Win=0 Len=0
113	2703.577094	10.0.1.11	10.0.2.22	TCP	66	51336→7	[ACK] Seq=514 Ack=514 Win=0 Len=0

▶ Frame 104: 74 bytes on wire (5880 bits) captured on interface 0

▼ Transm.
Source: 10.0.1.11
Destination: 10.0.2.22
[Stream index: 5]
[TCP Segment Len: 0]
Sequence number: 0 (relative sequence number)
Acknowledgment number: 0
Header Length: 40 bytes
▶ Flags: 0x002 (SYN)
Window size value: 29200
[Calculated window size: 29200]
Checksum: 0xd0d8 [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
▼ Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
▶ Maximum segment size: 1460 bytes
▶ TCP SACK Permitted Option: True
▶ Timestamps: TSval 3025644, TSecr 0
▶ No-Operation (NOP)
▶ Window scale: 7 (multiply by 128)

A TCP SYN, ACK Packet

104	2703.569100	10.0.1.11	10.0.2.22	TCP	74	51336→7	[SYN] Seq=0 Win=29200 Len=0
105	2703.570099	10.0.2.22	10.0.1.11	TCP	74	7→51336	[SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
106	2703.570420	10.0.1.11	10.0.2.22	TCP	66	51336→7	[ACK] Seq=1 Ack=1 Win=29200 Len=0
107	2703.571851	10.0.1.11	10.0.2.22	ECHO	578		Request
108	2703.573041	10.0.2.22	10.0.1.11	TCP	66	7→51336	[ACK] Seq=1 Ack=513 Win=0 Len=0
109	2703.573120	10.0.2.22	10.0.1.11	ECHO	578		Response
110	2703.573416	10.0.1.11	10.0.2.22	TCP	66	51336→7	[ACK] Seq=513 Ack=513 Win=0 Len=0
111	2703.575219	10.0.1.11	10.0.2.22	TCP	66	51336→7	[FIN, ACK] Seq=513 Ack=513 Win=0 Len=0
112	2703.576755	10.0.2.22	10.0.1.11	TCP	66	7→51336	[FIN, ACK] Seq=513 Ack=513 Win=0 Len=0
113	2703.577094	10.0.1.11	10.0.2.22	TCP	66	51336→7	[ACK] Seq=514 Ack=514 Win=0 Len=0

▶ Frame 105: 74 bytes on wire (588 bytes captured) on interface 0

▼ Transm:

Source

Destination port: 51336

[Stream index: 5]

[TCP Segment Len: 0]

Sequence number: 0 (relative sequence number)

Acknowledgment number: 1 (relative ack number)

Header Length: 40 bytes

▶ Flags: 0x012 (SYN, ACK)

Window size value: 28960

[Calculated window size: 28960]

Checksum: 0xa311 [unverified]

[Checksum Status: Unverified]

Urgent pointer: 0

▼ Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale

▶ Maximum segment size: 1460 bytes

▶ TCP SACK Permitted Option: True

▶ Timestamps: TSval 3026223, TSecr 3025644

▶ No-Operation (NOP)

▶ Window scale: 7 (multiply by 128)

▶ [SEQ/ACK analysis]

A TCP ACK Packet (last in 3 way handshake)

No.	Time	Source	Destination	Protocol	Length	Info
104	2703.569100	10.0.1.11	10.0.2.22	TCP	74	51336→7 [SYN] Seq=0 Win=29200 Len=0
105	2703.570099	10.0.2.22	10.0.1.11	TCP	74	7→51336 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
106	2703.570420	10.0.1.11	10.0.2.22	TCP	66	51336→7 [ACK] Seq=1 Ack=1 Win=29200 Len=0
107	2703.571851	10.0.1.11	10.0.2.22	ECHO	578	Request
108	2703.573041	10.0.2.22	10.0.1.11	TCP	66	7→51336 [ACK] Seq=1 Ack=513 Win=0 Len=0
109	2703.573120	10.0.2.22	10.0.1.11	ECHO	578	Response
110	2703.573416	10.0.1.11	10.0.2.22	TCP	66	51336→7 [ACK] Seq=513 Ack=513 Win=0 Len=0
111	2703.575219	10.0.1.11	10.0.2.22	TCP	66	51336→7 [FIN, ACK] Seq=513 Ack=513 Win=0 Len=0
112	2703.576755	10.0.2.22	10.0.2.22		66	7→51336 [FIN, ACK] Seq=513 Ack=513 Win=0 Len=0
113	2703.577094	10.0.1.11			66	51336→7 [ACK] Seq=514 Ack=514 Win=0 Len=0

▼ Trans

Sou

Destination Port: 7

[Stream index: 5]

[TCP Segment Len: 0]

Sequence number: 1 (relative sequence number)

Acknowledgment number: 1 (relative ack number)

Header Length: 32 bytes

► Flags: 0x010 (ACK)

Window size value: 229

[Calculated window size: 29312]

[Window size scaling factor: 128]

Checksum: 0x4219 [unverified]

[Checksum Status: Unverified]

Urgent pointer: 0

▼ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

► No-Operation (NOP)

► No-Operation (NOP)

► Timestamps: TSval 3025644, TSecr 3026223

► [SEQ/ACK analysis]

A TCP Data Packet (Ping)

	No.	Time	Source	Destination	Protocol	Length	Info
SYN Process	104	2703.569100	10.0.1.11	10.0.2.22	TCP	74	51336→7 [SYN] Seq=0 Win=29200 Len=0
	105	2703.570099	10.0.2.22	10.0.1.11	TCP	74	7→51336 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
	106	2703.570420	10.0.1.11	10.0.2.22	TCP	66	51336→7 [ACK] Seq=1 Ack=1 Win=29200 Len=0
	107	2703.571851	10.0.1.11	10.0.2.22	ECHO	578	Request
Ping	108	2703.573041	10.0.2.22	10.0.1.11	TCP	66	7→51336 [ACK] Seq=1 Ack=513 Win=0 Len=0
	109	2703.573120	10.0.2.22	10.0.1.11	ECHO	578	Response
	110	2703.573416	10.0.1.11	10.0.2.22	TCP	66	51336→7 [ACK] Seq=513 Ack=513 Win=0 Len=0
	111	2703.575219	10.0.1.11	10.0.2.22	TCP	66	51336→7 [FIN, ACK] Seq=513 Ack=513 Win=0 Len=0
FIN Process	112	2703.576755	10.0.2.22	10.0.1.11	TCP	66	7→51336 [FIN, ACK] Seq=513 Ack=513 Win=0 Len=0
	113	2703.577094	10.0.1.11	10.0.2.22	TCP	66	51336→7 [ACK] Seq=514 Ack=514 Win=0 Len=0

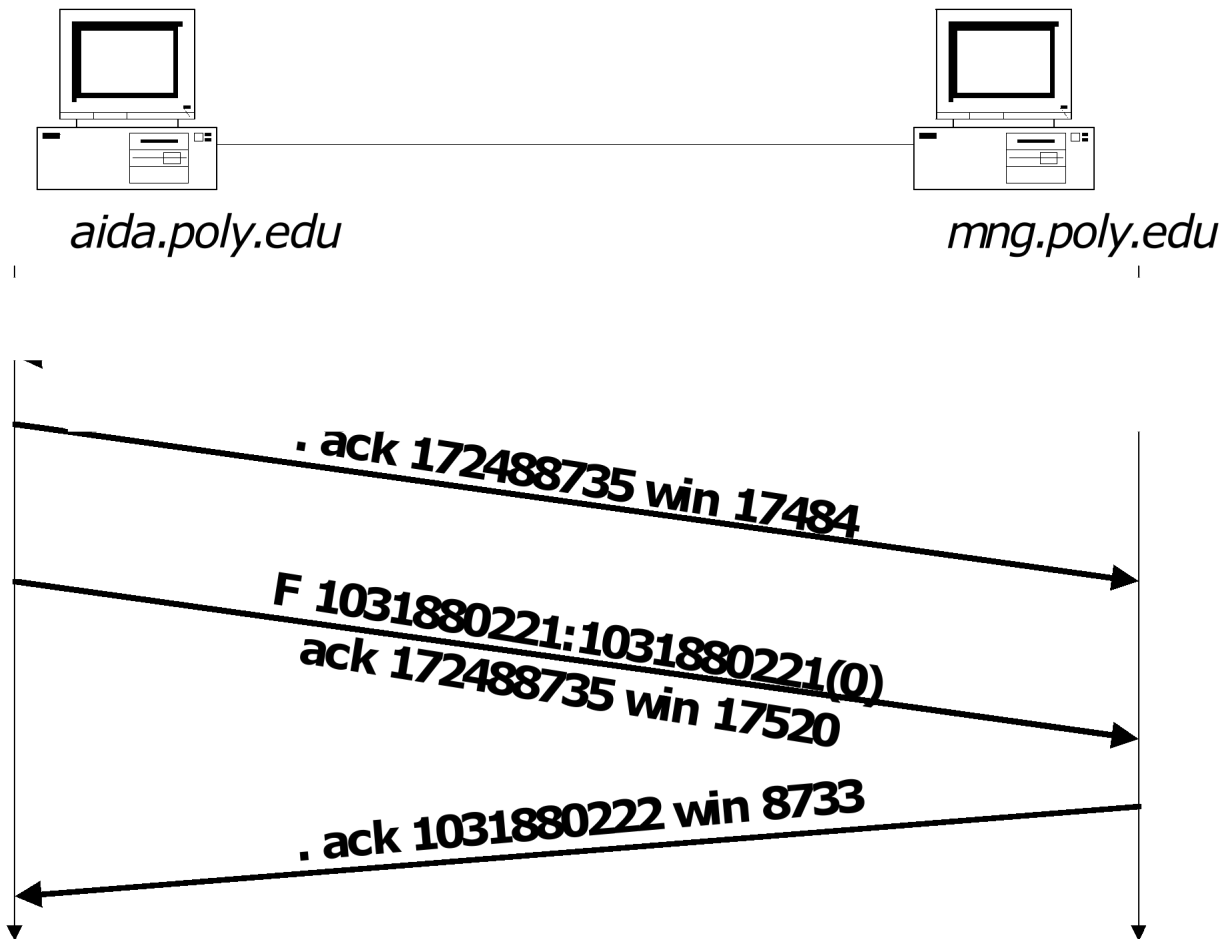
```

▶ Inter
▼ Trans
  Source Port: 51336
  Destination Port: 7
  [Stream index: 5]
  [TCP Segment Len: 512]
  Sequence number: 1 (relative sequence number)
  [Next sequence number: 513 (relative sequence number)]
  Acknowledgment number: 1 (relative ack number)
  Header Length: 32 bytes
  ▶ Flags: 0x018 (PSH, ACK)
  Window size value: 229
  [Calculated window size: 29312]
  [Window size scaling factor: 128]
  Checksum: 0x1d7e [unverified]
  [Checksum Status: Unverified]
  Urgent pointer: 0
  ▼ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
    ▶ No-Operation (NOP)
    ▶ No-Operation (NOP)
    ▶ Timestamps: TSval 3025645, TSecr 3026223
    ▶ [SEQ/ACK analysis]
  ▶ Echo
  
```

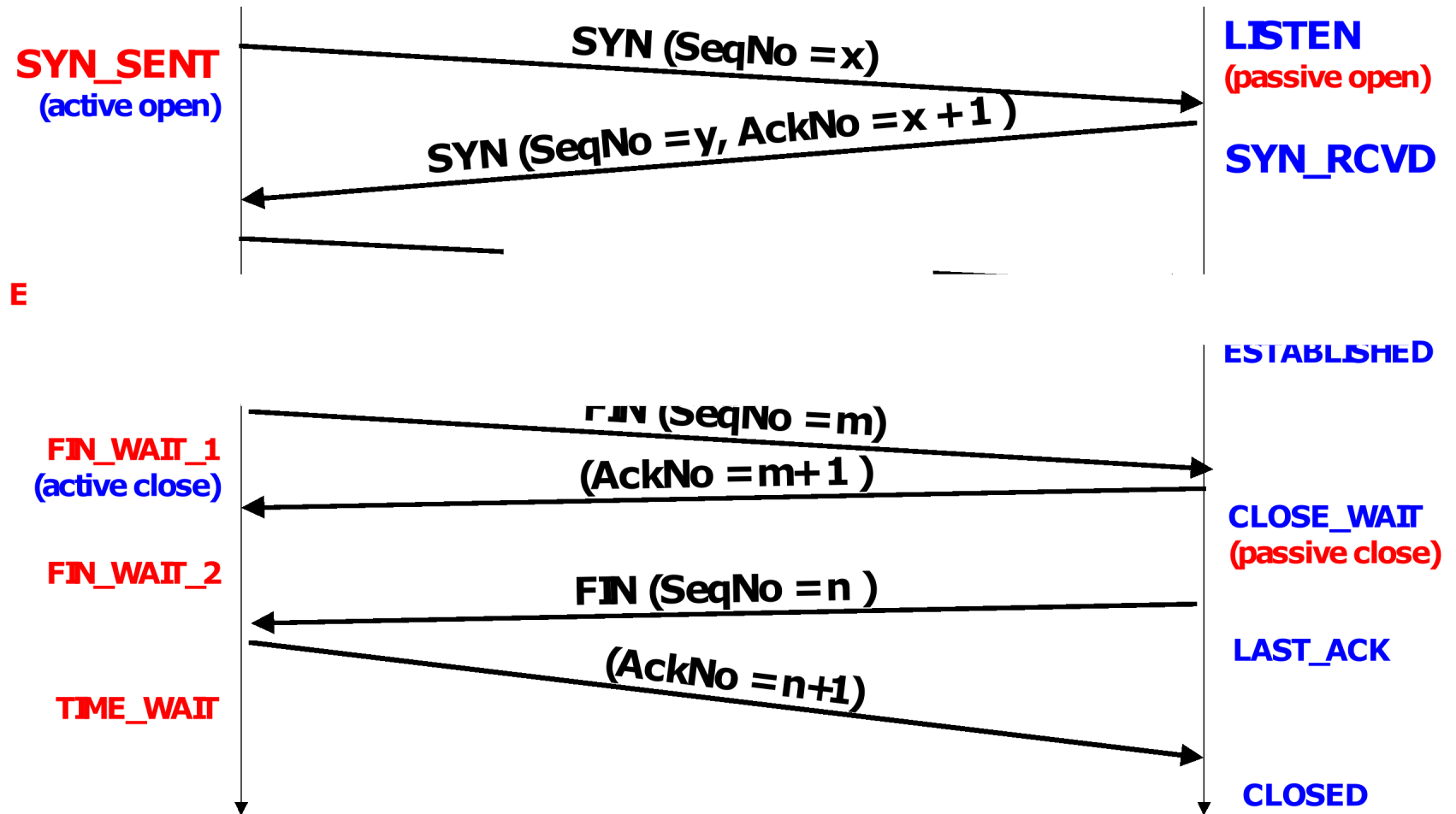
TCP Connection Termination

- Each end of the data flow must be shut down independently (“**half-close**”)
- If one end is done it sends a FIN segment. This means that no more data will be
- Four step
 - (1) X sends a FIN to Y (**active close**)
 - (2) Y ACKs the FIN,
(at this time: Y can still send data to X)
 - (3) and Y sends a FIN to X (**passive close**)
 - (4) X ACKs the FIN.

TCP Connection Termination



TCP States in “Normal” Connection Lifetime



TCP States

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for Ack
SYN_SENT	A connection request has been sent; wait for Ack
ESTABLISHED	Normal data transfer state
FIN_WAIT_1	Client has said it is finished
FIN_WAIT_2	Server has agreed to release
TIMED_WAIT	Wait for pending packets ("2MSL wait state")
CLOSING	Both Sides have tried to close simultaneously
CLOSE_WAIT	Server has initiated a release
LAST_ACK	Wait for pending packets

TIME_WAIT state

TIME_WAIT = 2MSL

- When TCP is in active close state and sends the final ACK, the connection **must stay in the TIME_WAIT state for twice the maximum seg**

2M

- Why?
TCP is given a chance to **resend** the final ACK. (Server will timeout after sending the first FIN segment and resend the FIN)

Resetting Connections

- Resetting connections is done by setting the RST flag
- **When the RST is flag set**
 - Indicates to Abort (Terminate) a connection